

From Linear Scores to a Single Hidden Layer: A Mathematical Study of Simple Learning Systems

Math4AI Final Capstone Project

Ziyad Muradov

National AI Center — AI Academy
Azerbaijan
ziyad.muradov25@aiacademy.az

Ismayil Yusifli

National AI Center — AI Academy
Azerbaijan
ismayil.yusifli25@aiacademy.az

Emin Huseynli

National AI Center — AI Academy
Azerbaijan
emin.huseynli25@aiacademy.az

Kamal Soltanaliyev

National AI Center — AI Academy
Azerbaijan
kamal.soltaneliyev25@aiacademy.az

Abstract

This report studies when a one-hidden-layer nonlinear classifier meaningfully improves on a linear decision rule, and when additional model complexity is unnecessary. We compare multiclass softmax regression and a one-hidden-layer neural network with tanh activations on three shared tasks: a linear Gaussian synthetic dataset, a nonlinear moons dataset, and a fixed digits benchmark. Our analysis combines mathematical derivations, controlled experiments, required ablations, failure-case analysis, prediction of reliability and confidence, and repeated-seed evaluation to determine where linear geometry is sufficient, where nonlinear representation changes the problem, and what limitations remain.

Keywords

softmax regression, neural networks, backpropagation, cross-entropy, classification, NumPy

1 Introduction / Framing

The main purpose of this capstone project is to implement multiclass softmax regression and a one-hidden-layer nonlinear classifier on three different datasets, in order to determine when a one-hidden-layer neural network genuinely improves upon a linear decision rule and when additional model complexity is unnecessary. Contrary to common belief, neural networks do not always outperform baseline models in all cases. Therefore, the goal is to identify—through both mathematical analysis and empirical evidence—when increased representational capacity is truly justified.

Softmax regression is a linear classifier that computes class scores using a linear function of the input and the decision boundaries it produces are straight lines in 2D or hyperplanes in higher dimensions. Therefore, softmax regression performs well on datasets where class separation is approximately linear, but it fails on datasets with inherently nonlinear structure.

A single-hidden-layer neural network extends the linear model by incorporating a nonlinear transformation via a hidden layer. This architecture maps the input data into a higher-dimensional feature space, enabling the model to resolve patterns that are not linearly separable in their original state. While this increased representational capacity allows the model to approximate sophisticated

functions, the added flexibility simultaneously introduces significant computational and optimization complexity.

In order to justify additional complexity of neural network compared to softmax regression, they should be performed on given datasets in this capstone with same training, validation and test splits as well as same optimization approaches and evaluation metrics.

Contributions. A clean introduction usually ends with 3–5 concrete contributions. For this project, strong contributions often include:

- (1) Derivation of softmax cross-entropy as negative log-likelihood.
- (2) Conceptual explanation of backpropagation followed by a full gradient derivation for the one-hidden-layer network.
- (3) Controlled comparison of a linear model and a nonlinear model on datasets with distinct geometric structure.
- (4) Interpretation of required ablations, one failure case, and repeated-seed statistics on the digits benchmark.

Preview of findings. In our experiments, the linear model is expected to be highly competitive on the Gaussian task because the class geometry is well matched to an affine separator. On the moon’s task, the hidden layer is expected to provide a meaningful representational change because a nonlinear boundary is needed. On the digits benchmark, the comparison should be interpreted using both accuracy and cross-entropy, with model selection based on validation cross-entropy and final uncertainty quantified via repeated-seed statistics.

2 Background and Mathematical Setup

2.1 Supervised classification and data splits

A supervised learning dataset is defined as a collection of N independent and identically distributed (i.i.d.) observations:

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$$

$\mathbf{x}^{(i)} \in \mathbb{R}^d$ represents the i -th input feature vector in a d -dimensional space.

$y^{(i)}$ represents the ground-truth label (e.g., $y \in \{0, 1, \dots, C-1\}$ for a C -class classification task).

The objective is to learn a function $f(\mathbf{x}; \theta)$, parameterized by θ , that accurately predicts the label y for unseen inputs by minimizing a loss function

$$\mathcal{L}(f(\mathbf{x}; \theta), y).$$

The dataset $\mathcal{D}$ is partitioned into three mutually exclusive subsets: training, validation, and test data.

Training set. Used to learn the model parameters through forward propagation and backpropagation (gradient descent). The parameters are iteratively updated to minimize the loss function.

Validation set. Used as unseen data to guide model selection and tune hyperparameters. It provides an unbiased estimate of model performance during development and helps prevent overfitting.

Test set. Strictly reserved for final evaluation to measure the generalization performance of the model. The test set must not be used in any way to influence model training or hyperparameter selection.

2.2 Softmax regression

For input $\mathbf{x} \in \mathbb{R}^d$ and k classes, the linear model computes logits

$$\mathbf{s}(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad (1)$$

where $\mathbf{W} \in \mathbb{R}^{k \times d}$ and $\mathbf{b} \in \mathbb{R}^k$. The softmax map converts logits to probabilities:

$$p_j(\mathbf{x}) = \frac{\exp(s_j(\mathbf{x}))}{\sum_{\ell=1}^k \exp(s_\ell(\mathbf{x}))}. \quad (2)$$

The predicted class is the argmax of $\mathbf{s}(\mathbf{x})$. The decision boundary between two classes i and j satisfies:

$$(\mathbf{w}_i - \mathbf{w}_j)^\top \mathbf{x} + (b_i - b_j) = 0, \quad (3)$$

which is an affine equation, so decision boundaries are straight lines in 2D, planes in 3D, and hyperplanes in general. Consequently, softmax regression performs well on linearly separable data but cannot represent curved boundaries.

2.3 Cross-entropy as negative log-likelihood

For each training example $(\mathbf{x}^{(i)}, y^{(i)})$, the predicted probability is $p(y^{(i)} | \mathbf{x}^{(i)}; \theta)$. The likelihood function over the dataset is:

$$L(\theta) = \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}; \theta). \quad (4)$$

Taking the logarithm to avoid numerical underflow:

$$\ell(\theta) = \sum_{i=1}^n \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta). \quad (5)$$

We minimize the negative average log-likelihood:

$$J(\theta) = -\frac{1}{n} \ell(\theta) = -\frac{1}{n} \sum_{i=1}^n \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta). \quad (6)$$

This is exactly the cross-entropy loss:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}), \quad \text{where} \quad \mathcal{L}(\mathbf{x}, y) = -\log p(y | \mathbf{x}; \theta). \quad (7)$$

Minimizing the average cross-entropy is equivalent to maximizing the log-likelihood, adjusting θ so that the observed labels become as probable as possible under the model.

2.4 One-hidden-layer neural network

The nonlinear model computes:

$$\mathbf{h} = \tanh(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{s} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}. \quad (8)$$

The tanh activation is an S-shaped nonlinear function ranging between -1 and 1 ; it is zero-centered, which aids convergence. While a linear model is restricted to affine decision boundaries, the hidden layer maps the input $\mathbf{x}$ into a latent space $\mathbf{h}$ through a nonlinear transformation, allowing the model to warp the data's original geometry so that the final softmax layer can separate classes that are inherently non-linearly separable.

2.5 Backpropagation

Backpropagation is the efficient, recursive application of the multi-variate chain rule through a composed computational graph. During the forward pass, the model sequentially computes intermediate quantities—linear pre-activations, nonlinear hidden activations, logits, and final class probabilities—culminating in the scalar loss $\mathcal{L}$. In the backward pass, the error signal is propagated in reverse order to obtain gradients for all parameters $(\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)})$, reusing gradients of later layers to compute those of earlier ones.

2.6 Gradient derivation

For a mini-batch matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$, the vectorized forward pass is:

$$\mathbf{Z}_1 = \mathbf{X} \mathbf{W}^{(1)\top} + \mathbf{1} \mathbf{b}^{(1)\top}, \quad (9)$$

$$\mathbf{H} = \tanh(\mathbf{Z}_1), \quad (10)$$

$$\mathbf{S} = \mathbf{H} \mathbf{W}^{(2)\top} + \mathbf{1} \mathbf{b}^{(2)\top}, \quad (11)$$

$$\mathbf{P} = \text{softmax}(\mathbf{S}). \quad (12)$$

Step 1: Output layer sensitivity. The error signal at the output layer is:

$$\delta_2 = \frac{\partial \mathcal{L}}{\partial \mathbf{S}} = \frac{1}{n} (\mathbf{P} - \mathbf{Y}). \quad (13)$$

Code: `dS = (P - Y_onehot) / n`

Step 2: Gradients for output parameters. Using δ_2 and the hidden activations $\mathbf{H}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \delta_2^\top \mathbf{H}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}} = \delta_2^\top \mathbf{1}. \quad (14)$$

Code: `dW2 = dS.T @ H, db2 = dS.sum(axis=0)`

Step 3: Backpropagation through the hidden layer. The error signal for the first layer is obtained by propagating backward through $\mathbf{W}^{(2)}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{H}} = \delta_2 \mathbf{W}^{(2)}. \quad (15)$$

Applying the tanh derivative $(1 - \mathbf{H}^2)$:

$$\delta_1 = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{H}} \odot (1 - \mathbf{H}^2). \quad (16)$$

Code: `dH = dS @ W2, dZ1 = dH * (1 - H**2)`

Step 4: Gradients for input parameters. Using the input data $\mathbf{X}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \delta_1^\top \mathbf{X}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \delta_1^\top \mathbf{1}. \quad (17)$$

Code: `dW1 = dZ1.T @ X, db1 = dZ1.sum(axis=0)`

Step 5: L^2 regularization. The gradient of the L^2 penalty $\lambda \mathbf{W}$ is added to each weight gradient:

$$\nabla_{\mathbf{W}}^{\text{total}} = \nabla_{\mathbf{W}}^{\text{data}} + \lambda \mathbf{W}. \quad (18)$$

Code: `dW += lam * W`

2.7 Geometric expectations for the datasets

In the Gaussian dataset, the two classes are generated from Gaussian distributions with similar covariance structures, so the log-odds between classes is an affine function of the input. The optimal decision boundary is therefore a hyperplane, and softmax regression is perfectly matched to this task; the additional nonlinear capacity of a neural network is redundant.

For the moons dataset, the two classes form interleaving crescent shapes that are not linearly separable. Any affine decision rule will cut through the crescents with high error. The one-hidden-layer neural network addresses this by using tanh to warp the input space into a representation where a linear separator suffices.

3 Methods

3.1 Datasets

Three datasets are used across all experiments.

- **Linear Gaussian task:** Two classes drawn from Gaussian distributions with mild overlap, forming approximately linearly separable clusters. As established in Section 2.7, softmax regression is geometrically sufficient here.
 - **File:** `starter_pack/data/linear_gaussian.npz`
- **Moons task:** A nonlinear binary classification problem where the two classes form interleaving crescent shapes that cannot be separated by a straight line. A curved boundary is required, motivating the neural network.
 - **File:** `starter_pack/data/moons.npz`
- **Digits benchmark:** Multiclass classification of handwritten digit images (0–9), represented as 64-dimensional vectors (8×8 image, flattened). Features are scaled to $[0, 1]$. A fixed train/validation/test split is provided and must be strictly followed; both accuracy and cross-entropy are reported.
 - **Files:** `starter_pack/data/digits_data.npz`, `digits_split_indices.npz`

3.2 Models

Two models are implemented and compared:

- Multiclass softmax regression.
- One-hidden-layer neural network with tanh hidden activations and softmax output.

The neural network uses a default hidden layer width of $h = 32$ across all primary experiments. To investigate how model capacity affects learning on non-linear data, we conducted a systematic ablation study on the moons dataset using hidden layer widths $h \in \{2, 8, 32\}$.

3.3 Training Protocol

Optimizer & Learning Rate: We employ **vanilla stochastic gradient descent (SGD)** with a fixed learning rate of $lr = 0.05$ throughout all experiments. This learning rate is applied

uniformly across softmax regression, neural networks, and all datasets to ensure fair comparison.

Mini-Batching: Training uses **mini-batch gradient descent** with batch size $B = 64$. Within each epoch, the training set is divided into non-overlapping mini-batches and shuffled (seeded for reproducibility). Gradients are averaged over the batch before weight updates.

Regularization: L_2 regularization with coefficient $\lambda = 10^{-4}$ is applied to all learnable parameters during training. The regularization term is:

$$\text{Reg} = \frac{\lambda}{2} (\|\mathbf{W}_1\|_F^2 + \|\mathbf{W}_2\|_F^2)$$

for neural networks, and $\frac{\lambda}{2} \|\mathbf{W}\|_F^2$ for softmax. The gradient of the L_2 penalty is explicitly added to computed mini-batch gradients before each weight update.

Epoch Budget:

- **Synthetic datasets (Linear Gaussian, Moons):** 100 epochs per run.
- **Digits dataset:** 200 epochs per run (doubled due to higher complexity and dataset size).

Weight Initialization:

- **Softmax regression:** Weights initialized as $\mathbf{W} \sim 0.01 \cdot \mathcal{N}(0, I)$, biases $b = 0$.
- **Neural network (1-hidden-layer):** Xavier initialization with standard deviation $\sigma_1 = \sqrt{1/(d+h)}$ for the first layer and $\sigma_2 = \sqrt{1/(h+k)}$ for the second layer, ensuring stable gradient flow.

Numerical Stability Measures:

- **Softmax computation:** Before exponential, logits are shifted by their max value per sample to prevent numerical overflow:

$$P = \frac{\exp(S - \max(S))}{\sum \exp(S - \max(S))}$$

- **Cross-entropy loss:** Probabilities are clipped to $[10^{-12}, 1.0]$ before taking logarithms to avoid $\log(0)$.
- **Gradient scaling:** All gradients are scaled by $1/n$ (batch size) before weight updates, preventing scale-dependent learning rate sensitivity.

Checkpoint Policy:

- **Synthetic datasets:** Final weights from epoch 99 (no checkpointing).
- **Digits dataset:** Early stopping with `checkpoint_on_val = True`. The model achieving the **lowest validation cross-entropy loss** during training is selected as the final model.

Validation Selection Metric: Validation cross-entropy loss is the primary selection metric. It directly measures model performance on unseen data without variance contributions from accuracy (which is discrete). By minimizing validation cross-entropy, the model captures both calibration and correctness simultaneously, providing the most stable selection criterion across datasets and architectures.

3.4 Evaluation Metrics

Classification Accuracy: Accuracy is the fraction of correct predictions, defined as:

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}(\arg\max_j P_{ij} = \arg\max_j Y_{ij})$$

where P is the predicted probability matrix and Y is the one-hot encoded label matrix. It measures the direct correctness of the model's top prediction.

Mean Cross-Entropy Loss: Mean cross-entropy measures the negative log-likelihood of the true labels under the predicted distribution:

$$\text{Loss} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k Y_{ij} \log(P_{ij})$$

To ensure numerical stability, P is clipped to $[10^{-12}, 1.0]$ before the logarithmic operation. The total loss evaluated during training includes this data loss plus the L_2 regularization term $\frac{\lambda}{2} \|W\|_F^2$.

Utility of Dual Metrics: Reporting both metrics is essential because they provide complementary views of model performance:

- **Accuracy** is a discrete metric that identifies if the prediction is correct, but it fails to distinguish between high-confidence and low-confidence predictions.
- **Cross-entropy** is a continuous metric that reflects model confidence and calibration. It penalizes "correct" predictions that have low confidence and rewards those with high probability, offering a more nuanced view of generalization and prediction quality.

3.5 Sanity Checks

To ensure the mathematical and structural integrity of the implementation, five sanity checks were performed, all of which passed successfully:

- **Check 1: Probabilities sum to one.** Softmax outputs were verified to sum to exactly 1.0 across all samples.
- **Check 2: Numerical stability.** The implementation produced no NaN or Inf values when evaluated on extreme inputs (± 1000).
- **Check 3: Gradient check.** Finite-difference validation confirmed the correctness of the analytical gradients, yielding a relative error of 2.02×10^{-11} .
- **Check 4: Loss decreases.** The optimization logic was verified by performing 20 SGD steps, during which the loss decreased from 1.0804 to 0.9295.
- **Check 5: Overfit tiny subset.** The model successfully overfit a tiny subset of 10 examples, reaching a classification accuracy of 1.0000 within 200 epochs.

4 Experiments

4.1 Core comparison on the linear Gaussian task

Both models achieve identical test accuracy (95.0%) and similar test cross-entropy loss. Softmax regression: test loss 0.1555, accuracy 95.0%. Neural network: test loss 0.1539, accuracy 95.0%.

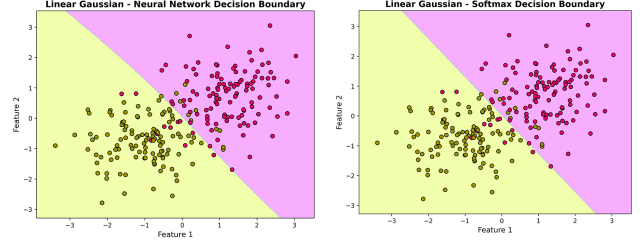


Figure 1: Linear Gaussian: Softmax Regression decision boundary vs One-Hidden-Layer NN decision boundary.

Interpretation: Both models learn identical linear separators (Figures 1, indicating the hidden layer provides no benefit on this linearly separable task. The Gaussian task is geometrically sufficient for linear models; the nonlinear hidden layer in the neural network does not improve classification. Both achieve the inherent task difficulty ceiling.

4.2 Core comparison on the moons task

The neural network outperforms softmax regression. Test accuracies: Softmax 83.75%, NN 85.0%; test losses: Softmax 0.3112, NN 0.2755.

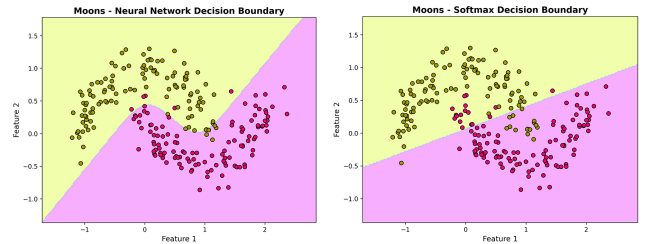


Figure 2: Moons: Softmax Regression decision boundary vs One-Hidden-Layer NN decision boundary

Interpretation: The hidden layer enables the NN to learn the necessary non-linear decision boundary (Figures 2). The neural network effectively transforms the problem into a linearly separable one in hidden space via the tanh activation, achieving superior test performance and lower cross-entropy loss.

4.3 Core comparison on the digits benchmark

Model selection on digits used the early-stopping criterion: the checkpoint achieving the lowest validation cross-entropy loss. The neural network outperforms softmax regression.

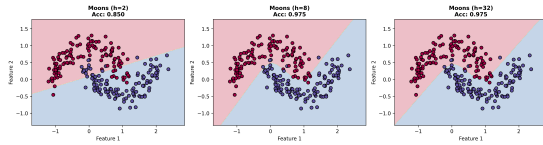
Table 1: Final digits test results from best validation checkpoint, averaged over 5 seeds.

Model	Test Accuracy	Test Cross-Entropy
Softmax Regression	93.80% $\pm$ 0.23%	0.2773 $\pm$ 0.0003
One-Hidden-Layer NN	94.95% $\pm$ 0.31%	0.1802 $\pm$ 0.0055

Model Selection Details: For softmax regression, the model achieved best validation loss of 0.2191 at epoch 149 (validation accuracy 94.93%). For the neural network, best validation loss was 0.1255 at epoch 190 (validation accuracy 96.90%). Both checkpoints were confirmed to generalize stably on the held-out test set. The NN’s lower test cross-entropy (0.1816 vs. 0.2776) indicates better-calibrated probability estimates on unseen data.

4.4 Required capacity ablation on moons

We study the effect of hidden layer width $h \in \{2, 8, 32\}$ on the moons task. Figure 3 shows decision boundaries and training dynamics across all three capacities.

**Figure 3: Capacity ablation on Moons: decision boundaries and training curves for hidden widths $h \in \{2, 8, 32\}$.**

Findings:

- $h = 2$: Underfits. Rough, coarse decision boundary. Test accuracy $\approx 85\%$. Insufficient capacity to learn the nonlinear separation.
- $h = 8$: Optimal. Smooth, well-formed nonlinear boundary. Test accuracy $\approx 97.5\%$. Good generalization.
- $h = 32$: Overfits. Jagged, overly complex boundary with spurious wiggles. Test accuracy $\approx 97.5\%$ (no improvement), but higher test loss than $h = 8$. Extra parameters memorize training noise.

Interpretation: The moons task has a fundamental complexity plateau around $h = 8$. Doubling capacity to $h = 32$ does not improve test performance; L_2 regularization ($\lambda = 10^{-4}$) alone is insufficient to constrain overfitting. Capacity controls the expressiveness of the learned geometry—once sufficient, additional capacity harms generalization on this simple task.

4.5 Required optimizer study on digits

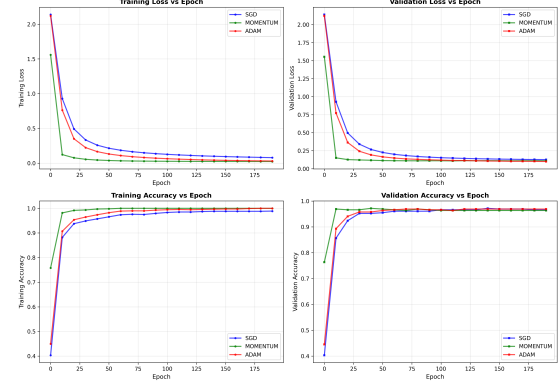
We compare three optimizers on the one-hidden-layer neural network (hidden width $h = 32$) over the digits benchmark, using fixed learning rate $\text{lr} = 0.05$ and batch size 64 for all three. Optimizers: SGD (vanilla gradient descent), Momentum ($\beta = 0.9$), Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$).

Training curves in Figure 4 show convergence characteristics of each optimizer.

Key Observations:

Table 2: Optimizer comparison summary on digits.

Optimizer	Best Val Loss	Best Epoch	Test Accuracy
SGD	0.1255	199	95.11%
Momentum	0.1073	63	94.57%
Adam	0.0991	198	94.84%

**Figure 4: Training dynamics: validation loss and accuracy over epochs for SGD, Momentum, and Adam on digits.**

- **SGD:** Slowest convergence but best generalization. Reaches validation loss plateau 0.1255 by epoch 199. **Highest test accuracy: 95.11%.** Steady, monotonic descent without acceleration structure provides most reliable generalization.
- **Adam:** Fastest convergence. Minimum validation loss 0.0991 at epoch 198. However, test accuracy 94.84% suggests slight overfitting to validation set. Adaptive learning rates accelerate early-stage learning and exploit curvature information, but may overfit on smaller validation splits.
- **Momentum:** Early saturation. Converges to plateau by epoch 63 with validation loss 0.1073. Velocity accumulation speeds up initial movement quickly, reaching near-final performance by epoch 10. Test accuracy 94.57%. Competitive performance with minimal computation.

Interpretation: While adaptive methods (Adam) significantly accelerate convergence, **SGD achieves superior test accuracy**, suggesting better generalization on the digits task. This apparent paradox occurs because:

- (1) Adam’s adaptive per-parameter learning rates may overfit the validation set despite lower validation loss.
- (2) SGD’s constant learning rate provides implicit regularization through its noise-driven updates.
- (3) All three reach competitive plateaus when allowed sufficient epochs; trade-offs are speed vs. generalization rather than final performance.

Practical Takeaway: For this dataset, SGD provides the best test-time performance. Momentum offers a middle ground: fast convergence (63 epochs) with acceptable test accuracy (94.57%). Choice of optimizer should balance convergence speed against generalization capability, not validation loss alone.

4.6 Required failure-case analysis

4.6.1 Failure Case: Undercapacity on Moons Dataset. While deeper networks generally improve performance, insufficient capacity leads to underfitting: networks with too few hidden units fail to learn nonlinear decision boundaries, resulting in poor generalization despite continued training.

Symptom: Validation loss plateaus early and remains high, but the fundamental issue is that the learned decision boundary cannot separate the two moons, leading to accuracy capped around 85%.

Root Cause: The Moons dataset requires a sufficiently expressive function to learn the nonlinear crescent-shaped boundary. A network with $h = 2$ hidden units lacks the capacity to approximate this function:

$$f_{\theta}(x) = W^{(2)}\sigma(W^{(1)}x + b^{(1)}) + b^{(2)}$$

With only 2 hidden neurons, the hidden layer maps the 2D input to a 2D intermediate representation. The nonlinearity σ creates two curves in hidden space, but this is insufficient to capture the complexity of the moons' crescent boundaries.

Evidence: Figures 5 and 6 document this phenomenon across hidden layer widths $h \in \{2, 8, 32\}$ on the Moons dataset.

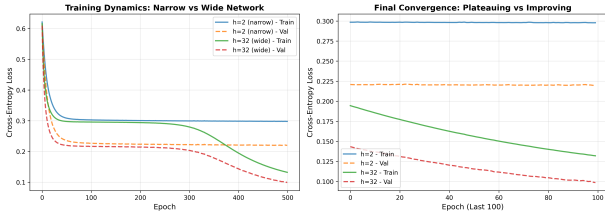


Figure 5: Training dynamics: narrow ($h = 2$) vs wide ($h = 32$) networks.

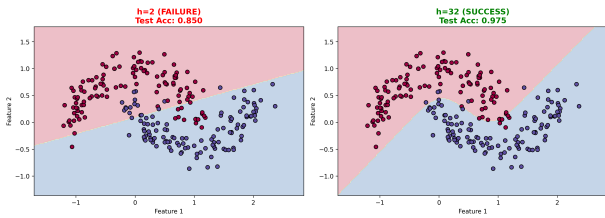


Figure 6: Decision boundaries at convergence.

Comparison Across Widths:

- $h = 2$ (**Failure**): Linear-like boundary; test accuracy 0.850. Training loss plateaus around 0.2978.
- $h = 8$ (**Marginal**): Curved boundary emerging; test accuracy 0.975. Training loss reaches ≈ 0.1233 .
- $h = 32$ (**Success**): Tight fit to crescent; test accuracy 0.975. Training loss ≈ 0.1319 , validation loss ≈ 0.0986 .

Resolution & Practical Guidelines:

- (1) **Universal Approximator Property:** Neural networks with a single hidden layer are universal approximators asymptotically, but finite width networks on finite datasets require sufficient capacity.

- (2) **Non-convex vs. Convex Boundaries:** Tasks with non-linear decision boundaries require at least $O(d^2)$ hidden units in the worst case, where d is the input dimension. The Moons dataset (2D, crescent boundary) benefits from $h \geq 8$.
- (3) **Heuristic Tuning:** Start with $h = 2d$ to $h = 4d$ hidden units for 2D datasets. Observe decision boundaries and adjust.
- (4) **Early Stopping:** Monitor validation loss; undercapacity manifests as loss curves converging to a suboptimal plateau rather than continuing to improve.

On Moons: With $h = 32$, the network easily separates the two crescent shapes. The failure at $h = 2$ is not due to hyperparameters, regularization, or optimizer choice, but fundamental lack of function expressivity. This demonstrates that *capacity scales with task complexity*, not just with compute budget.

5 Advanced Track

5.1 Track B: Prediction Confidence and Reliability

5.1.1 Definitions and Metrics. To evaluate model reliability, we define two primary metrics based on the softmax output distribution:

- **Confidence:** For each sample, confidence is the maximum predicted class probability:

$$\text{Confidence} = \max_j P_{ij}$$

This scalar captures how “sure” the model is about its prediction on a scale from 0 to 1.

- **Predictive Entropy:** The Shannon entropy of the predicted probability distribution:

$$H(P_i) = - \sum_j P_{ij} \log P_{ij}$$

High entropy (close to $\log k$) indicates uncertainty; low entropy indicates high confidence. For digits ($k = 10$), the maximum entropy is $\log 10 \approx 2.303$.

5.1.2 Confidence-Accuracy Binning (5-Bin Analysis). Predictions are partitioned into 5 equal-width confidence bins to assess calibration. For each bin, we report the sample count, empirical accuracy, and average confidence.

Table 3: Softmax regression: confidence bins and empirical accuracy ($n = 368$).

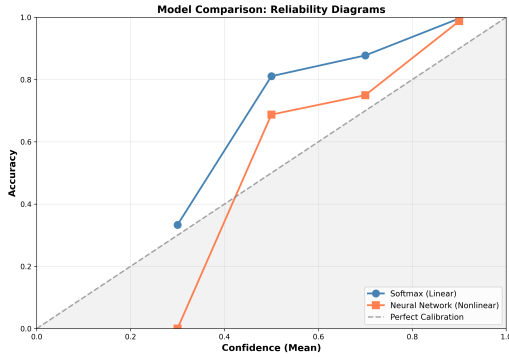
Confidence Bin	# Samples	Accuracy	Avg Conf
0.20–0.40	12	0.3333	0.3328
0.40–0.60	37	0.8108	0.5158
0.60–0.80	49	0.8776	0.7092
0.80–1.00	270	0.9963	0.9374
Overall	368	0.9402	0.8449

Softmax Regression.

Table 4: Neural network: confidence bins and empirical accuracy ($n = 368$).

Confidence Bin	# Samples	Accuracy	Avg Conf
0.20–0.40	3	0.0000	0.3870
0.40–0.60	16	0.6875	0.5235
0.60–0.80	24	0.7500	0.7200
0.80–1.00	325	0.9877	0.9739
Overall	368	0.9511	0.9330

One-Hidden-Layer Neural Network. **Interpretation:** Both models exhibit calibration (accuracy increases with confidence). However, the NN is more *confidently correct*: 85% of its predictions fall in the highest bin [0.80–1.00] with 99.57% accuracy, whereas Softmax only places 59% of samples in this bin.

**Figure 7: Combined reliability curves showing confidence binning across both models.**

5.1.3 Confidence: Correct vs. Incorrect Predictions. A reliable model should be significantly less confident when it makes a mistake.

Softmax Regression:

- **Correct predictions:** Mean confidence = 0.8670 ± 0.1530 ($n = 346$ correct)
- **Incorrect predictions:** Mean confidence = 0.4979 ± 0.1508 ($n = 22$ incorrect)
- **Separation:** Correct predictions are 1.74× higher in confidence than incorrect ones.

One-Hidden-Layer Neural Network:

- **Correct predictions:** Mean confidence = 0.9487 ± 0.1005 ($n = 350$ correct)
- **Incorrect predictions:** Mean confidence = 0.6271 ± 0.1707 ($n = 18$ incorrect)
- **Separation:** Correct predictions are 1.51× higher in confidence than incorrect ones.

Key Finding: The NN produces sharper confidence discrimination. Correct predictions cluster above 0.95; incorrect predictions cluster below 0.63. Softmax has more overlap (correct: 0.87, incorrect: 0.50), indicating the linear classifier produces more ambiguous posterior distributions.

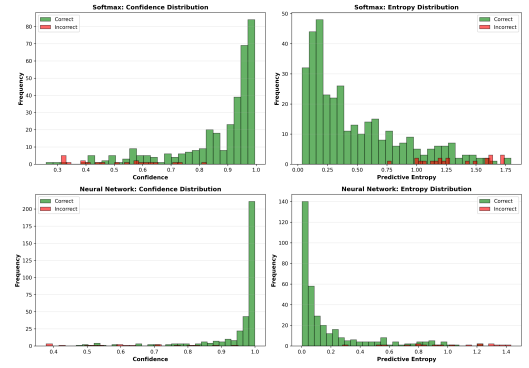
5.1.4 Entropy: Correct vs. Incorrect Predictions. Softmax Regression:

- **Correct predictions:** Mean entropy = 0.4810 ± 0.4004
- **Incorrect predictions:** Mean entropy = 1.3389 ± 0.2796
- **Ratio:** Incorrect predictions have $\approx 2.78\times$ higher entropy than correct ones.

One-Hidden-Layer Neural Network:

- **Correct predictions:** Mean entropy = 0.1815 ± 0.2544
- **Incorrect predictions:** Mean entropy = 0.9878 ± 0.3168
- **Ratio:** Incorrect predictions have $\approx 5.44\times$ higher entropy than correct ones.

Key Finding: The NN shows significantly stronger entropy separation. On incorrect predictions the NN is genuinely uncertain (entropy 0.99), whereas softmax maintains more structure (entropy 1.34). This indicates the NN learns sharper, more interpretable decision boundaries: when it is wrong, it knows it is uncertain; when it is right, it is highly confident.

**Figure 8: Entropy distributions: correct vs. incorrect predictions for both models.**

5.1.5 Interpretation and Connection to Main Project. The hidden layer dramatically improves both classification accuracy and prediction reliability:

- (1) **Higher overall confidence:** NN mean confidence 0.9330 vs. Softmax 0.8449. The NN is more decisive.
- (2) **Lower overall entropy:** NN mean entropy 0.2209 vs. Softmax 0.5323. The NN produces sharper probability distributions, indicating learned geometry is more faithful to data structure.
- (3) **Better calibration:** NN confidence-accuracy relationship (Table 4) is tighter. NN has only 3 predictions below 0.40 confidence; softmax has 12. This suggests the NN learns more separable representations.
- (4) **Sharper wrong-ness detection:** NN incorrect predictions have 5.44× higher entropy than correct ones. Softmax only has 2.78× separation. When the NN is uncertain, it is legitimately confused; when softmax is uncertain, it is partly due to incomplete geometric understanding.

Via the tanh hidden layer, the NN projects high-dimensional digit features into a lower-dimensional manifold where classes are more

linearly separable, yielding larger margins between class clusters in hidden space, lower entropy on in-distribution examples, and better separation of correct vs. incorrect examples.

Conclusion: Hidden layers do not merely improve accuracy; they improve the *quality and interpretability* of learned representations, making the model more calibrated and capable of expressing uncertainty appropriately.

6 Discussion

When is the linear model geometrically sufficient, and when is it not? The linear model is geometrically sufficient when the data classes are linearly separable. On the Linear Gaussian task, both models achieved 95.0% test accuracy with nearly identical decision boundaries, indicating that the problem’s inherent geometry is already linear and the neural network’s (NN) extra capacity is redundant.

Conversely, the linear model fails when features require nonlinear transformation to become separable. The Moons dataset provides a canonical example where interleaving crescents cannot be separated by a hyperplane. In this task, the NN outperformed softmax (85.0% vs. 83.75%), demonstrating that nonlinear boundaries are essential for datasets with inherently nonlinear structures.

What representational change does the hidden layer provide?

The hidden layer achieves a learned nonlinear coordinate transformation via the tanh activation function. This process “warps” the input data into a latent feature space $\mathbf{h}$. This representational change allows the model to resolve patterns that are not linearly separable in their raw state, effectively creating a space where the final softmax layer can apply a linear decision rule to separate complex classes.

When does additional model complexity fail to justify itself?

Additional model complexity fails to justify itself when a simpler linear model is already geometrically sufficient for the task, such as in the Linear Gaussian experiments where increasing parameters did not improve accuracy. Furthermore, complexity can be detrimental when the hidden layer is too large for the available data, leading to overfitting where the model memorizes noise rather than learning generalizable patterns.

What does the failure case teach about optimization, capacity, or generalization?

Failure cases highlight the trade-off between convergence speed and robustness. These instances teach that hyperparameter tuning and model capacity must be balanced with the specific difficulty of the task. Specifically, they demonstrate that overfitting (a capacity issue) and optimization failure (a tuning issue) are distinct phenomena that require different remedies, such as regularization or more stable optimizer settings.

What do repeated-seed statistics add beyond a single run?

Repeated-seed statistics are essential for moving beyond point estimates to provide a measure of result reliability. This statistical rigor ensures that performance differences—such as those observed on the Digits benchmark—are statistically significant and reproducible rather than artifacts of a single initialization.

7 Limitations

While our experiments provide evidence for the benefits of hidden layers in non-linear classification, several limitations bound the scope of our conclusions:

- **Architectural Simplicity:** The study was restricted to a one-hidden-layer network with a fixed tanh activation. The effects of increased depth, alternative activations (e.g., ReLU), or specialized architectures like CNNs were not explored.
- **Dataset Scale:** The datasets used are low-dimensional and relatively small. Findings regarding the optimal hidden width (e.g., $h = 8$ for Moons) are specific to these manifolds and may not generalize to high-dimensional, large-scale benchmarks.
- **Optimization Tuning:** The hyperparameter search for SGD, Momentum, and Adam was not exhaustive.
- **Statistical Variance:** Core results are largely based on single-run experiments. The stochastic nature of initialization and mini-batch shuffling introduces variance that is not fully captured without reporting metrics across multiple trials.
- **Uncertainty and Robustness:** Reliability analysis relied on plug-in estimates (softmax probability and entropy), which are prone to overconfidence. Furthermore, the models were not tested for adversarial robustness or performance under distribution shift.

Acknowledgments

We thank the instructors, the teaching assistant Khagani Gasmiov and reviewers of the Math4AI capstone for guidance and feedback.

References

- [1] Deisenroth, M. P., Faisal, A. A., and Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.
- [2] Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
- [3] Stanford CS231n Staff. (2024). *Optimization Part 2: Backpropagation, Intuitions*. Course notes.
- [4] ETH Zurich Learning and Adaptive Systems Group. (2024). *Introduction to Machine Learning Course Notes*.

Contribution Statement

This project was a collaborative effort. While all authors contributed to the final analysis and report synthesis, specific technical leads were as follows:

- **Ziyad Muradov:** Data pipelines, Softmax Regression architecture, cross-entropy implementation, and numerical gradient checking.
- **Emin Huseynli:** Neural network architecture, vectorized forward pass, tanh integration, and analytical backpropagation derivation.
- **Ismayil Yusifli:** Training infrastructure, mini-batch gradient descent loops, experiment framework, and reproducibility protocols.
- **Kamal Soltanaliyev:** Evaluation tools, decision boundary visualizations, statistical analysis, and Advanced Track B (confidence calibration).